A PROJECT ON

Courier Service Management

SUBMITTED IN

PARTIAL FULFILLMENT OF THE REQUIREMENT

FOR THE COURSE OF DIPLOMA IN ADVANCED COMPUTING FROM CDAC

SUNBEAM INSTITUTE OF INFORMATION TECHNOLOGY

Hinjawadi

SUBMITTED BY:

Prashantkumar Murlidhar Dudhmal

UNDER THE GUIDENCE OF:

Mr. Snehal Jadhav

Faculty Member

Sunbeam Institute of Information Technology, Pune

ACKNOWLEDGEMENT

A project usually falls short of its expectation unless aided and guided by the right persons at the right time. We avail this opportunity to express our deep sense of gratitude towards Mr. Nitin Kudale (Center Coordinator, SIIT, Pune) and Mr. Yogesh Kolhe (Course Coordinator, SIIT ,Pune) .

We are deeply indebted and grateful to them for their guidance, encouragement and deep concern for our project. Without their critical evaluation and suggestions at every stage of the project, this project could never have reached its present form.

Last but not the least we thank the entire faculty and the staff members of Sunbeam Institute of Information Technology, Pune for their support.

Prashantkumar Murlidhar Dudhmal
0225 PG-DAC
SIIT Pune

CERTIFICATE

This is to certify that the project work under the title 'Courier Service Management' is done by Prashantkumar Murlidhar Dudhmal in partial fulfillment of the requirement for award of Diploma in Advanced Computing Course.

Mr. Snehal Jadhav	**Mr. Yogesh Kolhe**
Project Guide	**Course Co-Coordinator**

Date: 03/02/2026

1. INTRODUCTION TO PROJECT

The **Courier Service Management System** is a comprehensive web-based application developed to automate, streamline, and manage end-to-end courier service operations in a centralized digital environment. In today's fast-paced logistics industry, courier companies handle a large volume of parcels daily, making manual tracking, coordination, and management inefficient, error-prone, and time-consuming. This project addresses these challenges by providing a modern, technology-driven solution that enhances operational efficiency, accuracy, and transparency.

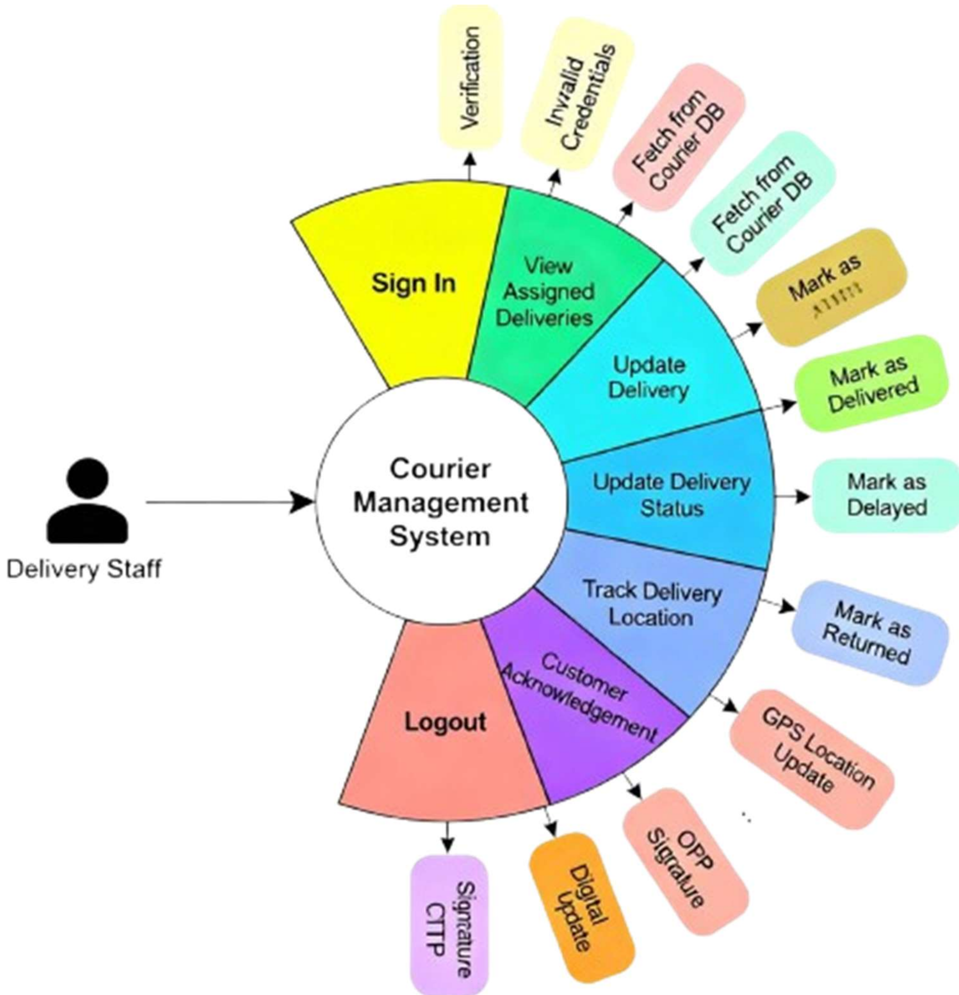
The primary objective of this system is to replace traditional manual courier handling processes with a **centralized, scalable, and efficient digital platform**. The system enables seamless management of parcel booking, shipment tracking, hub-to-hub transfers, delivery assignment, and administrative control. By automating these workflows, the application significantly reduces human intervention, minimizes operational delays, and improves customer satisfaction.

From the customer's perspective, the system offers an intuitive interface that allows users to **book courier parcels online**, **track shipments in real time**, and **view complete delivery history**. Customers receive timely updates on parcel status, ensuring transparency throughout the delivery lifecycle. This real-time tracking capability enhances trust and reliability, which are critical factors in courier and logistics services.

On the operational side, the system provides powerful tools for **administrators and staff** to manage courier hubs, delivery routes, vehicles, delivery personnel, and parcel statuses effectively. Admin users have centralized control over system configurations, user roles, hub management, and operational monitoring. Staff members can efficiently update shipment progress, manage deliveries, and coordinate hub-to-hub transfers, ensuring smooth and timely parcel movement across locations.

Overall, the Courier Service Management System provides a robust, secure, and scalable digital solution that modernizes courier operations, improves service quality, and supports efficient logistics management in a highly competitive environment.

2.REQUIREMENTS


Verification
Irvalid Credentials
Fetch from Courier DB
Fetch from Courier DB
Mark as
Mark as Delivered
Mark as Delayed
Mark as Returned
GPS Location Update
OPP Signature
Digital Update
Signature CTTP
Sign In
View Assigned Deliveries
Update Delivery
Update Delivery Status
Track Delivery Location
Customer Acknowledgement
Logout
Courier Management System
Delivery Staff


Staff Manager
Login
Verify 2FA
Manage Staff
Add New Staff
Update Staff Details
Remove Staff
Assign Deliveries
Allocate to Staff
Reassign Delivery
Monitor Deliveries
Track Progress
Handle Escalations
Investigate Complaints
Resolve Disputes
Resolve Rispots
Generate Reports
View Dashboard
Logout

2.1 FUNCTIONAL REQUIREMENTS

The functional requirements describe the core features and operations supported by the Courier Service Management System. These requirements define how different users interact with the system and how courier operations are performed digitally to ensure efficiency, security, and accuracy.

2.1.1 User Roles and Access Control

The system supports multiple user roles, each with clearly defined responsibilities and controlled access rights. A **role-based access control (RBAC)** mechanism is implemented to restrict system functionalities according to user roles, ensuring data security and operational clarity.

The supported roles include:

- **Admin**
- **Hub Manager**
- **Delivery Staff**
- **Customer**

Each user can access only those features that are relevant to their responsibilities, preventing unauthorized actions and protecting sensitive operational data.

2.1.2 Customer Module

The Customer Module allows end users to interact with the courier system in a simple and user-friendly manner. It provides all essential features required for booking and tracking courier parcels.

The system allows customers to:

- Register and log in securely to the application
- Book courier parcels by entering sender and receiver details
- Select parcel type, parcel weight, and delivery priority
- Generate a unique tracking ID after successful booking
- Track shipment status in real time using the tracking ID
- View booking history and detailed parcel status updates

This module enhances transparency and improves customer satisfaction by providing real-time visibility of courier deliveries.

2.1.3 Admin Module

The Admin Module provides centralized control over the entire courier management system. Admin users have full access to configure, monitor, and manage system-wide operations.

The system enables the admin to:

- Log in securely using authentication mechanisms
- Manage courier hubs by adding, updating, or deactivating hubs
- Manage vehicles and define delivery routes
- Create, update, and deactivate staff accounts
- Monitor all parcel movements across hubs and cities
- View system-wide analytics, reports, and performance statistics

This module ensures smooth administration, operational oversight, and effective decision-making.

2.1.4 Hub & Delivery Management

This module supports day-to-day operational activities at the hub and delivery level. It ensures efficient coordination between hubs and delivery personnel.

The system allows hub managers to:

- Manage parcels assigned to their respective hubs
- Assign parcels to delivery staff for last-mile delivery
- Track hub-to-hub parcel transfers
- Update parcel status such as *In Transit*, *Out for Delivery*, and *Delivered*

Delivery staff can:

- View assigned delivery tasks
- Update delivery status in real time
- Report delivery completion or issues

This module ensures timely parcel movement and accurate delivery status updates.

2.1.5 Parcel Tracking System

The Parcel Tracking System is a core feature that enables real-time monitoring of courier shipments throughout the delivery lifecycle.

The system provides:

- Automatic generation of a unique tracking ID for each parcel
- Real-time parcel status updates at every delivery stage
- Route-based parcel movement tracking between hubs
- Location-based tracking using city latitude and longitude coordinates

This system improves transparency, reduces customer inquiries, and enables efficient tracking of courier parcels across multiple locations.

2.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes of the Courier Service Management System. These requirements focus on usability, performance, security, and system constraints to ensure reliable and efficient operation of the application.

2.2.1 Interface Requirements

The system provides a modern, responsive, and user-friendly web interface that ensures smooth interaction across devices and user roles.

The interface requirements include:

- A responsive web interface that works efficiently on desktops, tablets, and mobile devices
- Role-based dashboards customized for Admin, Hub Manager, Delivery Staff, and Customer
- Simple and intuitive navigation for easy access to system features
- Secure authentication pages for login and registration to protect user credentials

The interface design focuses on clarity, accessibility, and ease of use to improve user experience.

2.2.2 Performance Requirements

The system is designed to deliver high performance and reliability under varying workloads.

The performance requirements include:

- Support for multiple concurrent users without system degradation
- Fast parcel search and tracking to provide real-time updates to customers
- Optimized API response times for smooth frontend-backend communication

Efficient database queries and optimized REST APIs ensure quick data retrieval and processing.

2.2.3 Security Requirements

Security is a critical aspect of the Courier Service Management System, as it handles sensitive user and operational data.
The system ensures security through:

- JWT-based authentication for secure and stateless user sessions
- Role-based authorization to restrict access based on user responsibilities
- Secure password encryption using industry-standard hashing algorithms
- Protected REST APIs to prevent unauthorized access and data manipulation

These mechanisms collectively ensure data confidentiality, integrity, and system protection.

2.2.4 Hardware & Software Requirements

- Hardware Requirements
- Minimum 4 GB RAM
- Intel i3 processor or above
- Stable internet connectivity
- Software Requirements
- Frontend: React.js
- Backend: Java, Spring Boot
- Database: MySQL
- Server: Apache Tomcat / Embedded Spring Boot Server
- Cloud Platform: AWS (EC2 for deployment, RDS for database – basic usage)
- Tools: Docker and Jenkins (basic exposure for containerization and CI/CD)

These requirements ensure smooth development, deployment, and scalability of the application.

3. DESIGN

3.1 Database Design

The following table structures depict the database design.

1. User Table

Column Name	Data Type	Description
user_id (PK)	INT	Unique user identifier
name	VARCHAR(100)	Full name
email	VARCHAR(100)	User email (unique)
password	VARCHAR(255)	Encrypted password
role	VARCHAR(30)	Admin / Hub Manager / Delivery Staff / Customer
phone	VARCHAR(15)	Contact number
created_at	TIMESTAMP	Account creation time

2. Customer Table

Column Name	Data Type	Description
customer_id (PK)	INT	Unique customer ID
user_id (FK)	INT	Reference to User table
address	TEXT	Customer address
city	VARCHAR(50)	City name
pincode	VARCHAR(10)	Postal code

3. Hub Table

Column Name	Data Type	Description
hub_id (PK)	INT	Unique hub identifier
hub_name	VARCHAR(100)	Hub name
city	VARCHAR(50)	Hub city
latitude	DOUBLE	City latitude
longitude	DOUBLE	City longitude
status	VARCHAR(20)	Active / Inactive

4. Staff Table

Column Name	Data Type	Description
staff_id (PK)	INT	Unique staff ID
user_id (FK)	INT	Reference to User table
hub_id (FK)	INT	Assigned hub
role	VARCHAR(30)	Hub Manager / Delivery Staff

5. Vehicle Table

Column Name	Data Type	Description
vehicle_id (PK)	INT	Vehicle ID
vehicle_number	VARCHAR(20)	Registration number
vehicle_type	VARCHAR(30)	Bike / Van / Truck
hub_id (FK)	INT	Assigned hub

6. Route Table

Column Name	Data Type	Description
route_id (PK)	INT	Route ID
source_hub_id (FK)	INT	Source hub
destination_hub_id (FK)	INT	Destination hub
distance_km	DOUBLE	Distance

7. Parcel Table

Column Name	Data Type	Description
parcel_id (PK)	INT	Parcel ID
tracking_id	VARCHAR(50)	Unique tracking ID
sender_id (FK)	INT	Customer (sender)
receiver_name	VARCHAR(100)	Receiver name
receiver_address	TEXT	Receiver address
weight	DOUBLE	Parcel weight
parcel_type	VARCHAR(30)	Document / Box
priority	VARCHAR(20)	Normal / Express
status	VARCHAR(30)	Booked / In Transit / Delivered

8. Parcel Movement Table

Column Name	Data Type	Description
movement_id (PK)	INT	Movement record ID
parcel_id (FK)	INT	Parcel reference
hub_id (FK)	INT	Current hub
status	VARCHAR(30)	In Transit / Out for Delivery
timestamp	TIMESTAMP	Update time

9. Delivery Assignment Table

Column Name	Data Type	Description
assignment_id (PK)	INT	Assignment ID
parcel_id (FK)	INT	Parcel reference
staff_id (FK)	INT	Delivery staff
assigned_date	DATE	Assignment date
delivery_status	VARCHAR(30)	Pending / Delivered

E-R Diagram,Dataflow diagram and Class Diagram:

Go to Appendix A

4. CODING STANDARDS IMPLEMENTED

The Courier Service Management System follows well-defined coding standards and best practices to ensure code quality, maintainability, scalability, and readability. Industry-standard Java and web development conventions have been adopted throughout the application.

Java Naming Conventions

Standard Java naming conventions are followed consistently across the project:

- Class names use **PascalCase** (e.g., ParcelService, UserController)
- Method and variable names use **camelCase** (e.g., trackParcel(), deliveryStatus)
- Constants use **UPPER_CASE** with underscores
- Package names are written in **lowercase**

These conventions improve code readability and maintain uniformity across modules.

Layered Architecture

The application follows a **layered architectural pattern**, ensuring a clear separation of responsibilities:

- **Controller Layer:** Handles HTTP requests and responses
- **Service Layer:** Contains business logic and validations
- **Repository Layer:** Manages database interactions using JPA/Hibernate

This structure improves maintainability, testing, and scalability of the system.

RESTful API Design

The backend is implemented using **RESTful web services**, following standard REST principles:

- Proper usage of HTTP methods such as **GET, POST, PUT, DELETE**
- Resource-oriented API endpoints
- Stateless communication using JSON data format

This design ensures smooth frontend-backend integration and scalability.

Exception Handling

Centralized exception handling is implemented using **global exception handlers**:

- Common and custom exceptions are handled uniformly
- Meaningful error messages are returned to the client
- Proper HTTP status codes are used for error responses

This approach improves system robustness and simplifies debugging.

Separation of Concerns

Each application component focuses on a specific responsibility:

- Controllers manage request flow
- Services handle business logic
- Repositories manage persistence logic

This **clean separation of concerns** reduces coupling and improves code clarity.

Meaningful Variable and Method Naming

Descriptive and meaningful names are used for variables, methods, and classes:

- Names reflect the purpose of the component
- Avoidance of ambiguous or short variable names

This practice enhances understandability and reduces maintenance effort.

Code Reusability and Modularization

The codebase is designed to maximize reusability and modularity:

- Common logic is abstracted into reusable services and utilities
- Modular components reduce duplication
- Easy integration of new features without affecting existing code

5. TEST REPORT

Testing was carried out to ensure that the Courier Service Management System functions correctly, reliably, and meets all specified requirements. Both functional and integration testing were performed to validate system behavior and component interaction.

5.1 Functional Testing

Functional testing was conducted to verify that each system feature works according to the defined functional requirements.

SR NO	TEST CASE	EXPECTED RESULT	ACTUAL RESULT
1	User Login	Successful login	Pass
2	Parcel Booking	Booking confirmation generated	Pass
3	Parcel Tracking	Correct parcel status displayed	Pass
4	Admin Dashboard	All system data loaded correctly	Pass
5	Delivery Update	Parcel delivery status updated	Pass

All functional test cases executed successfully without any critical defects.

5.2 Integration Testing

Integration testing was performed to verify smooth interaction between different system components and services.

The following integrations were tested:

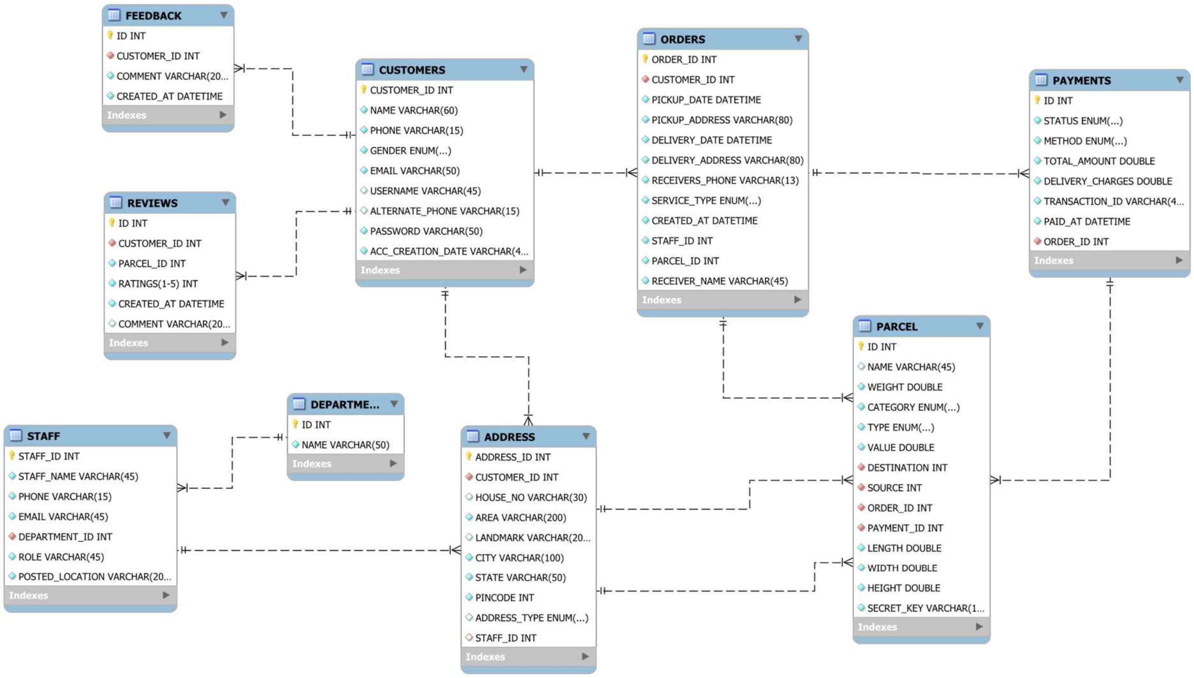
- **Frontend–Backend API integration** to ensure correct request and response handling
- **Database connectivity** to verify successful data storage and retrieval
- **Microservice communication** to validate seamless interaction between independent services

The system demonstrated stable integration across all components.

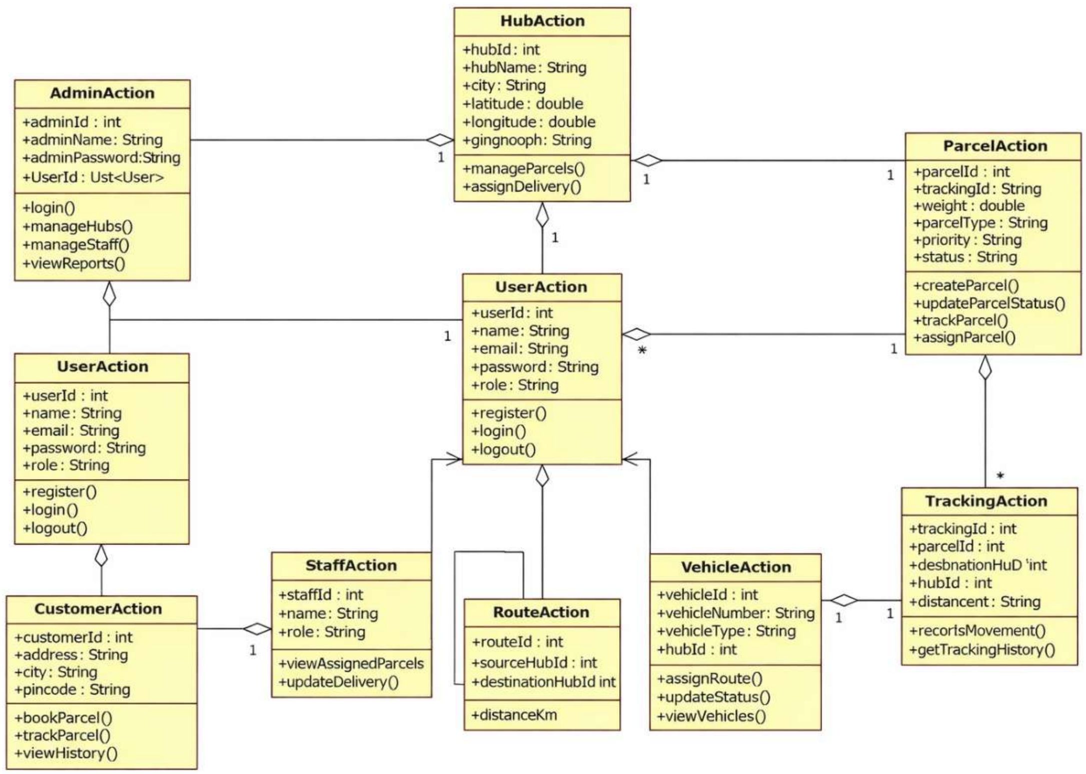
6. PROJECT MANAGEMENT RELATED STATISTICS

DATE	WORK PERFORMED	SDLC PHASE	ADDITIONAL NOTES
Jan 10, 2026	Project allotment and initial requirement discussion	Feasibility Study	Project scope and objectives were discussed with the guide
Jan 12, 2026	User requirements gathering and analysis	Requirement Analysis	Functional and non-functional requirements identified
Jan 14, 2026	Initial SRS document preparation and validation	Requirement Analysis (Elicitation)	SRS reviewed and refined based on feedback
Jan 16, 2026	Design of use cases, ER diagram, class diagram, and UI mockups	Design Phase	Database and system architecture finalized
Jan 18, 2026	Business logic and microservice design started	Design Phase	Service-level responsibilities defined
Jan 20, 2026	Backend development started (Spring Boot, APIs)	Implementation	Core services and REST APIs implemented
Jan 22, 2026	Frontend development started (React.js)	Implementation	UI components and dashboards developed
Jan 24, 2026	Backend–Frontend integration	Implementation	API integration and data flow testing
Jan 26, 2026	Unit testing and validation implementation	Testing	Input validations and error handling added
Jan 27, 2026	Module testing by team members	Testing (Module Testing)	Functional issues identified and resolved
Jan 28, 2026	System and integration testing	Testing (Integration Testing)	End-to-end workflow tested
Jan 29, 2026	Bug fixing and optimization	Debugging	Performance and logic issues resolved
Jan 30, 2026	Deployment and documentation	Deployment	Project prepared for final submission

Appendix A
Entity Relationship Diagram

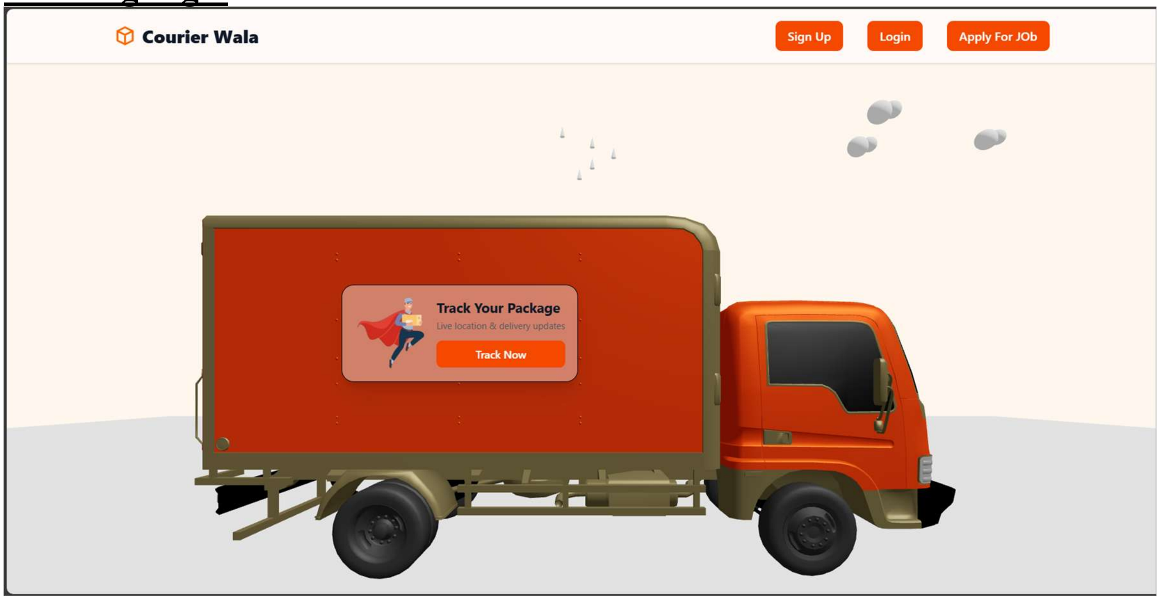


Class Diagram

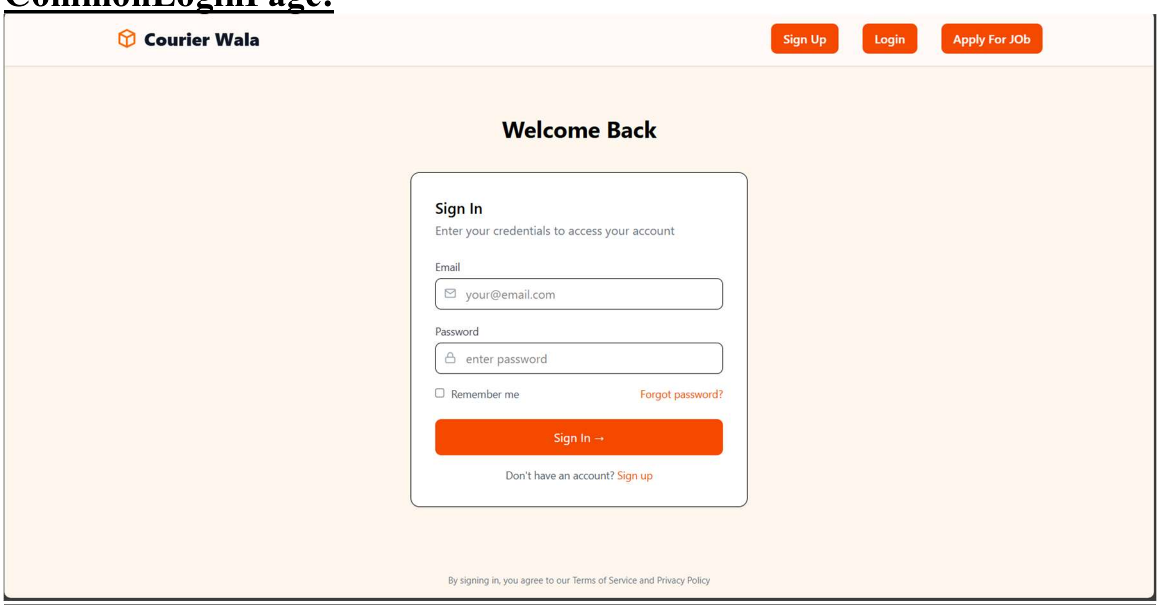


Appendix B

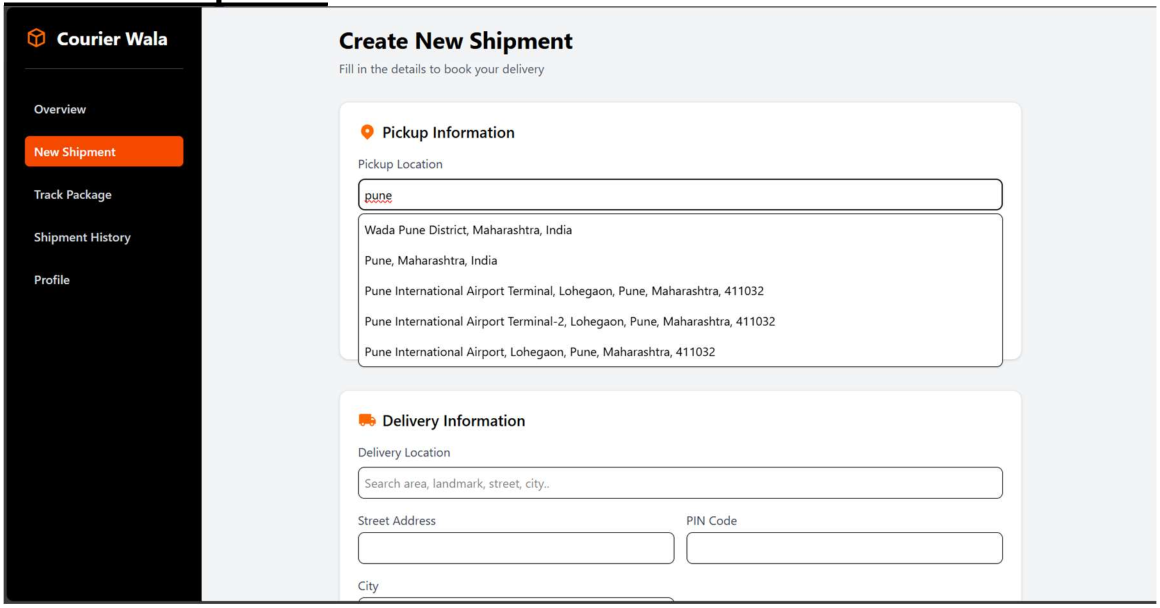
LandingPage:



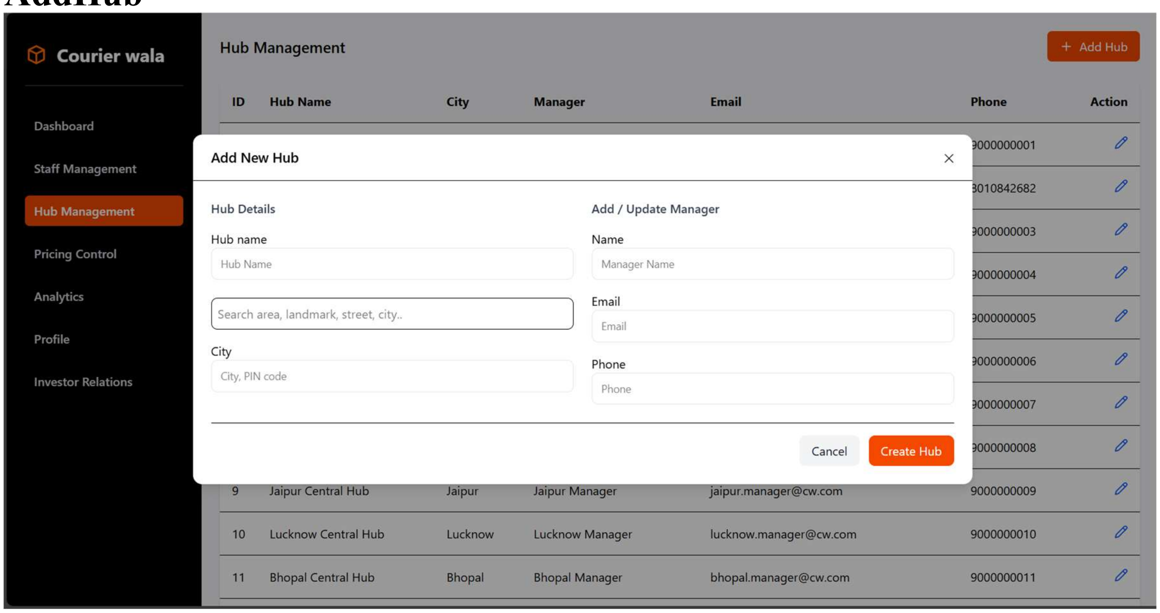
CommonLoginPage:



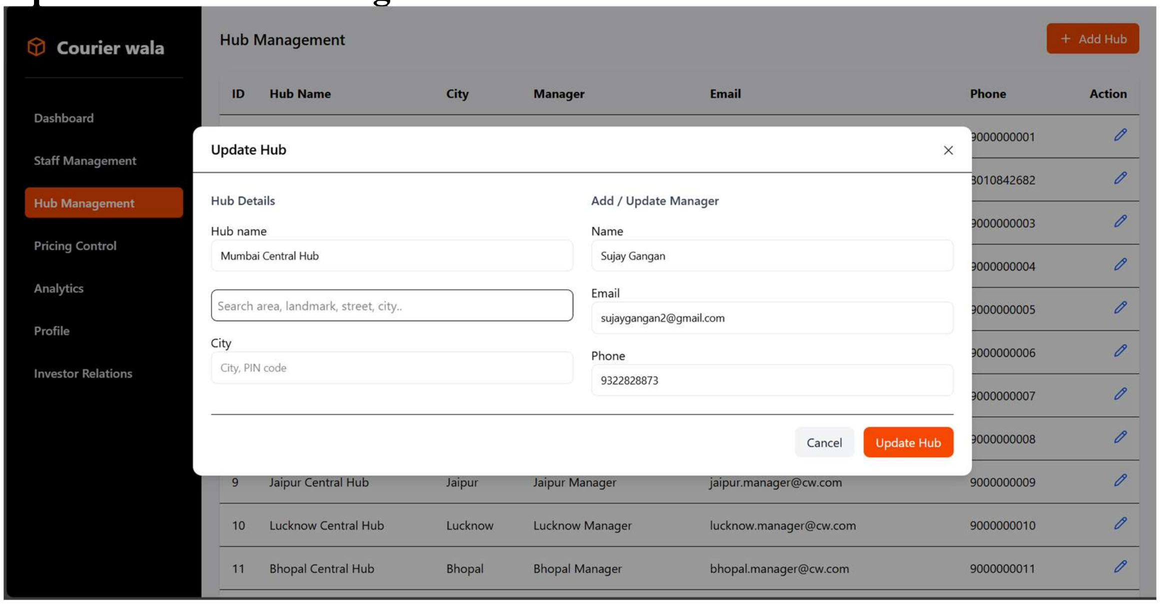
Add New Shipment:



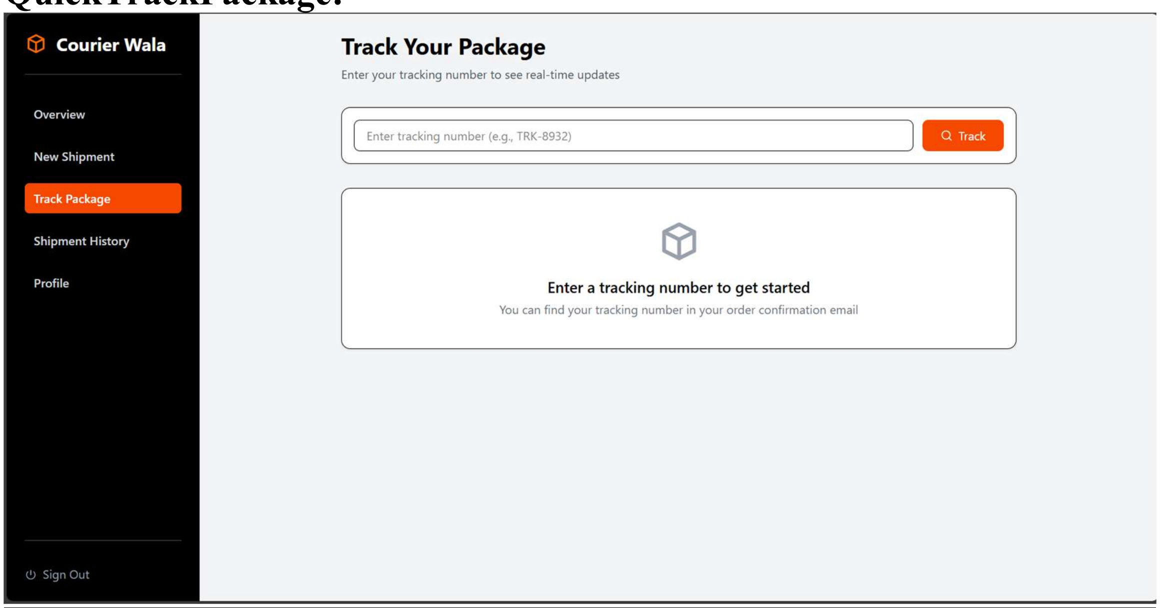
AddHub



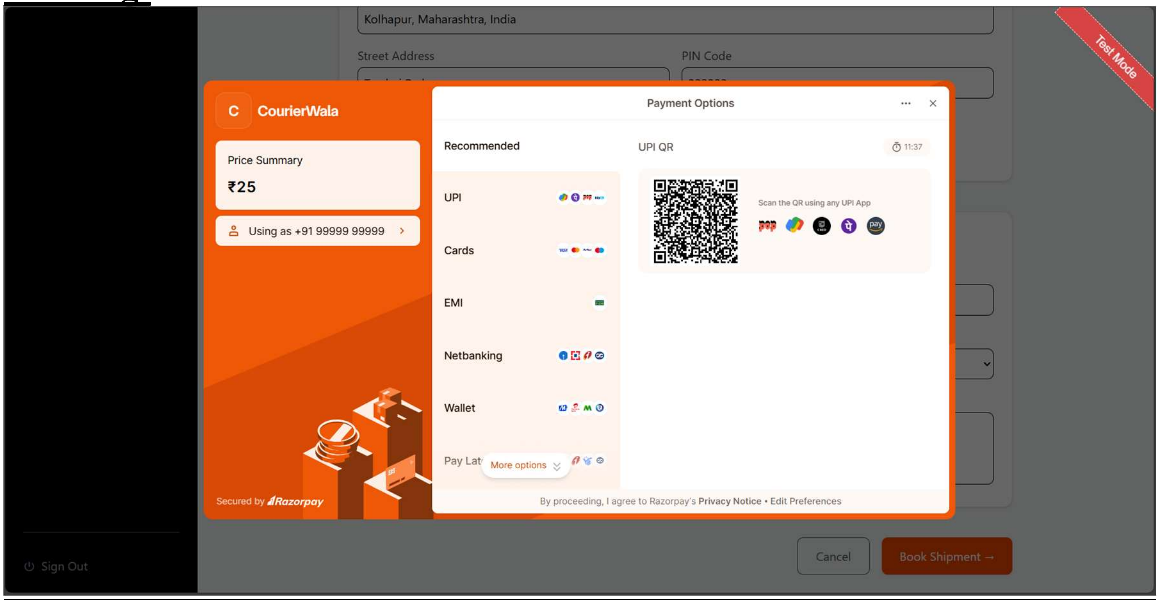
Update Hub and Manager



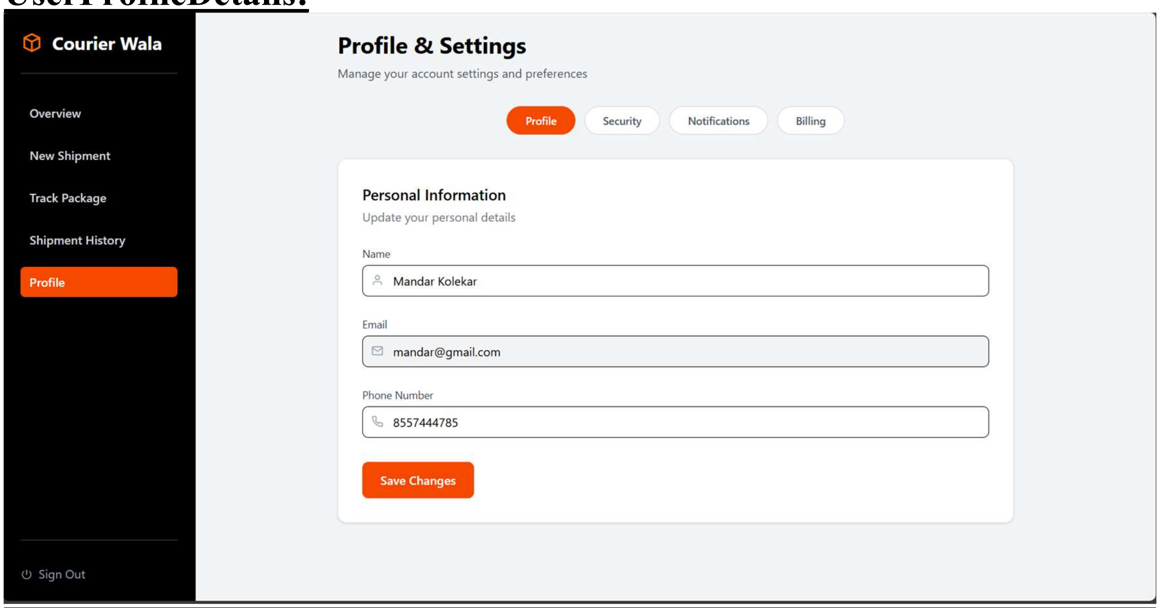
QuickTrackPackage:



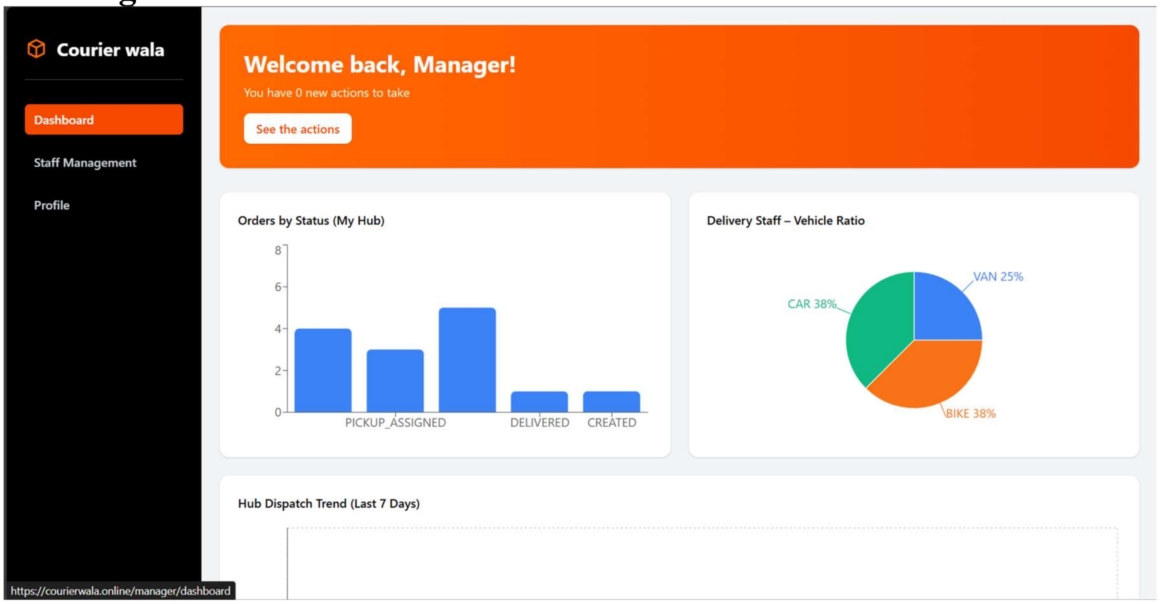
Banking:



UserProfileDetails:



BookingDetails:



7.REFERENCES:

http://www.google.com

https://www.delhivery.com/

https://docs.spring.io/spring-boot/index.html

https://React.dev